HashMap with Open Addressing & Tombstones

Open Addressing, Linear Probing, Tombstones, Automatic Resizing · Mentor-Led Lab

Developed by TalenciaGlobal

Difficulty ★★★★ Advanced

Problem

Design a MyHashMap that maps integer keys to integer values with open addressing and linear probing - the value-carrying cousin of the previous lab. Keys and values are kept in parallel arrays; a deleted key becomes a tombstone so probe sequences survive.

Support put(key, value) (insert or update), get(key) (the value or -1), and remove(key). Track size as the count of real entries (excluding tombstones), and resize automatically - doubling the capacity and re-inserting only valid keys - when the load factor passes 0.7.

Learning objectives: see how a HashMap stores keys and values together under open addressing, support deletion with tombstones, manage the load factor with resizing, and understand the foundation this lays for Robin Hood and concurrent hash maps.

Operations & Rules

- PUT key value - insert the pair, or update the value if the key exists.
- GET key - output the value, or -1 if the key is absent.
- REMOVE key - mark the key's slot as a tombstone if present.
- Parallel arrays hold keys and values; the key array uses empty (-1) and tombstone (-2) sentinels. Linear probing visits (hash + i) mod capacity.
- Load factor: resize to double the capacity whenever size / capacity exceeds 0.7 after an insertion; size counts real entries only.

Input / Output Format

Input: Line 1: the initial capacity. Each later line: PUT key value, GET key, or REMOVE key (read until end of input).

Output: One line per GET (its result), then Size and Capacity of the final table.

Examples

Example 1

Input: cap=4; put(1,100); put(5,200); get(1); remove(1); get(1); put(1,300); get(1)

Output:

```
100
-1
300
```

Explanation: 5 collides with 1 and probes to the next slot; after removing 1 a tombstone holds the slot, and re-putting 1 reuses that tombstone with the new value 300.

Example 2

Input: cap=4; put(2,20); get(2); put(2,25); get(2)

Output:

```
20
25
```

Explanation: The second put updates the existing key in place.

Constraints

- 0 <= key, value <= 10^6; up to 10,000 operations.
- Open addressing only - no built-in map types.
- Tombstones for deletion; resize keeps the load factor <= 0.7; size excludes tombstones.